

# Лекция 1. Точный поиск паттерна в тексте

# 1. Точный поиск паттерна в тексте

## 1.1 Префикс-функция

**Опр. 1 (Супрефикс)** Строка  $T$  называется *супрефиксом* строки  $S$ , если она является одновременно и префиксом, и суффиксом строки  $S$ .

Например, для строки «abacaba» супрефиксами являются строки «», «a», «aba» и сама «abacaba».

Примечание: пустая строка и сама строка  $S$  всегда являются её супрефиксами. Супрефикс, не равный всей строке, называется *собственным*.

**Опр. 2 (Префикс-функция)** Префикс-функцией от строки  $S$  называют такой массив  $\pi(S)$  длины  $|S| + 1$ , что  $\pi(S)[i]$  равно длине максимального собственного (то есть не равного всей строке) супрефикса строки  $S.substr(0, i)$ ,  $0 \leq i \leq |S|$ .

Примечание:  $S.substr(0, i)$  — это префикс строки  $S$  длиной  $i$  символов. У пустого префикса собственных супрефиксов нет, поэтому полагаем  $\pi[0] = 0$ .

**Алгоритм 1 (Алгоритм за линию)**

Заведём одну переменную  $k$  — длину текущего супрефикса-кандидата. Она объявляется один раз перед перебором всех  $i$  (изначально  $k = 0$ , так как  $\pi[0] = 0$ ). К моменту, когда мы хотим посчитать  $\pi[i + 1]$ , мы уже посчитали все  $\forall j \leq i \pi[j]$ , а в  $k$  уже лежит значение  $\pi[i]$ , оставшееся с предыдущего шага.

Отдельно: при  $i = 0$  шаги ниже не нужны —  $S[k]$  и  $S[i]$  это один и тот же символ ( $k = i = 0$ ), сравнивать нечего, и  $\pi[1] = 0$  всегда (у односимвольной строки нет непустой собственной границы).

1. Смотрим на символ  $S[k]$ . Если он равен  $S[i]$ , тогда значение  $\pi[i + 1]$  равно  $k + 1$  (так как мы продолжили и предыдущий супрефикс, и соответствующий ему суффикс, а значит длина супрефикса увеличилась на 1, также он не может увеличиться больше, чем на 1, так как иначе, убрав последний символ, мы бы увеличили  $\pi[i]$ , а мы знаем, что он уже посчитан правильно, противоречие). Увеличиваем  $k$  на 1 и переходим к следующему  $i$ .
2. Если  $S[k] \neq S[i]$ , мы смотрим на более короткий супрефикс — тот, что продолжает границу текущего кандидата, а его длина как раз  $\pi[k]$ . То есть мы меняем  $k = \pi[k]$  и рекурсивно продолжаем алгоритм на шаге 1 (для того же  $i$ ).
3. Если мы в итоге смогли завершиться на шаге 1, переходим к следующему  $i$ . Иначе мы скатились в 0 (то есть  $k = 0$ ) и тогда на шаге 1 сравниваем  $S[0]$  с  $S[i]$ : если равны,  $k$  становится 1 (это и есть  $\pi[i + 1]$ ), если нет —  $k$  остаётся 0 (это тоже  $\pi[i + 1]$ ).

### Время работы

На первый взгляд кажется, что раз на шаге 2 может быть много откатов подряд, то на одну позицию  $i$  алгоритм способен потратить много времени, значит суммарно выйдет не  $O(n)$ .

Но всего есть два вида событий, которые меняют  $k$ :

- Шаг 1 с совпадением —  $k$  увеличивается ровно на 1, и таких событий за весь алгоритм не больше  $n$  (по одному на каждый переход  $i \rightarrow i + 1$ ).
- Шаг 2 (откат) —  $k$  каждый раз строго уменьшается (так как  $\pi[k] < k$  при  $k > 0$ : супрефикс всегда короче самой строки, которую он продолжает).

Увеличений за весь алгоритм не больше  $n$ , значит и откатов (шагов 2) — тоже не больше  $n$ . Каждый откат — это  $O(1)$  работы, значит на все откаты суммарно уходит  $O(n)$ . А на сам просмотр  $S[k]$  для каждого  $i$  уходит ещё  $O(n)$  — по  $O(1)$  на каждый. Итого —  $O(n)$  на весь алгоритм.

### Реализация (C++)

```
void compute_prefix_function(const std::string& text, std::vector<int>& prefix) {
    int length = text.size();
    prefix.assign(length + 1, 0); // prefix[0] = prefix[1] = 0 — у односимвольной
                                // строки не бывает непустой собственной границы

    int border = 0;
    for (int index = 1; index < length; ++index) {
        while (true) {
            if (text[border] == text[index]) { // шаг 1
                ++border;
                break;
            }
            if (border == 0) { // шаг 3
                break;
            }
            border = prefix[border]; // шаг 2
        }
        prefix[index + 1] = border;
    }
}
```

**Алгоритм 2** (КМП (Кнут — Моррис — Пратт)) Задача: найти все вхождения образца  $t$  длины  $m$  в строку  $s$  длины  $n$ .

Решение: строим строку  $u = t\#s$ , где  $\#$  — символ-разделитель, не встречающийся ни в  $t$ , ни в  $s$ . Считаем на  $u$  префикс-функцию.

Разделитель  $\#$  необходим, чтобы чётко разделить строки  $t$  и  $s$  и в ходе подсчёта префикс-функции у нас супрефикс никогда не превысит  $m$ , так как на  $m+1$  месте стоит символ, не встречающийся нигде, кроме этой позиции.

Из-за этого нам достаточно посмотреть на значение префикс-функции: на всех местах, где записано число  $m$ , будут заканчиваться вхождения строки  $t$  в строку  $s$ .

Пример:  $t = \text{«aba»}$ ,  $s = \text{«abacaba»}$ .  $u = \text{«aba\#abacaba»}$ . Префикс-функция для этой строки —  $[0, 0, 0, 1, 0, 1, 2, 3, 0, 1, 2, 3]$ . Мы видим, что на позициях 7 и 11 стоят тройки, значит там заканчиваются вхождения строки  $t$ . Действительно:  $abacaba$  содержит  $aba$  на позициях 0 и 4.

### Время работы

Строка  $u$  имеет длину  $n + m + 1$ , а префикс-функция считается за линию от длины строки, поэтому весь поиск занимает  $O(n + m)$ .

## Реализация (C++)

```
void find_occurrences(const std::string& pattern, const std::string& source,
                     std::vector<int>& occurrences) {
    std::string combined = pattern + '#' + source;
    std::vector<int> prefix;
    compute_prefix_function(combined, prefix);
    int pattern_length = static_cast<int>(pattern.size());
    for (std::size_t index = 0; index < prefix.size(); ++index) {
        if (prefix[index] == pattern_length) {
            occurrences.push_back(static_cast<int>(index) - 2 * pattern_length - 1);
        }
    }
}
```

## 1.2 Z-функция

**Опр. 3 (Z-функция)** Z-функцией от строки  $S$  называют такой массив  $z(S)$  длины  $|S|$ , что  $z(S)[i]$  равно длине максимального префикса строки  $S[i:]$ , который одновременно является и префиксом всей строки  $S$ .

Примечание: обычно считается, что Z-функция в нулевой позиции равна 0.

**Опр. 4 (Z-блок)** Z-блоком назовём подстроку с началом в позиции  $i$  и длиной  $z[i]$ .

**Алгоритм 3 (Линейный алгоритм)**

Пусть переменная  $left$  - это левая граница самого правого Z-блока (если таких несколько, самого большого), а  $right$  - правая граница этого же Z-блока. Эти переменные будут обновляться по мере вычисления z-функции.

Пусть на текущем этапе мы хотим посчитать  $z[i]$ , и  $\forall j < i$  мы уже посчитали  $z[j]$ . Тогда есть 2 варианта:

- 1  $i > right$ : тогда пройдемся  $j$  от 0 и будем сравнивать  $S[i+j]$  и  $S[j]$ , пока они совпадают. Как только не совпало, останавливаемся: в  $z[i]$  кладем  $j$  (столько символов совпало в префиксах  $S$  и  $S.substr(i)$ ), а границы обновляем на  $left = i$ ,  $right = i + j - 1$  (это последний из совпавших символов).
- 2  $i \leq right$ : тогда мы знаем, что подстрока  $S[left...right]$  совпадает с префиксом  $S$ . Значит ответ для  $i$  начинается также, как и для  $i - left$  (так как  $i$  попадает в отрезок  $[left, right]$ , то есть  $S[0] = S[left]$ , ...,  $S[right - left] = S[right]$ ).
  - 1 Если  $i + z[i - left] \leq right$ , то есть, если у нас ответ для  $z[i - left]$  не выходит за границы уже посчитанного блока, тогда  $z[i] = z[i - left]$  (все значения мы уже посчитали ранее)
  - 2 Если же  $i + z[i - left] > right$ , то вплоть до  $right$  всё точно совпадает, так что просто продолжаем сравнивать  $S[i+j]$  и  $S[j]$  дальше, начиная с  $j = right - i + 1$  (а не с нуля — иначе досчитывали бы уже известное заново). Получившееся значение — это  $z[i]$ , границы обновляем так же, как в первом пункте.

## Время работы

На первый взгляд кажется, что раз в пункте 2.2 можно каждый раз досравнивать посимвольно, суммарно выйдет не  $O(n)$ . Но `right` за весь алгоритм только растёт и не может превысить  $n - 1$ , а каждое совпавшее сравнение (в пунктах 1 и 2.2) продвигает его на 1 — значит суммарно совпавших сравнений не больше  $n$ . А несовпавшее сравнение на каждой позиции  $i$  всего одно (сразу её останавливает). Итого —  $O(n)$ .

## Реализация (C++)

```
void compute_z_function(const std::string& text, std::vector<int>& z_function) {
    int length = text.size();
    z_function.assign(length, 0);
    int left = 0, right = 0;
    for (int index = 1; index < length; ++index) {
        int j = 0;
        if (index <= right) { // шаг 2
            if (index + z_function[index - left] <= right) { // шаг 2.1
                z_function[index] = z_function[index - left];
                continue;
            }
            j = right - index + 1; // шаг 2.2
        }
        while (index + j < length && text[index + j] == text[j]) { // шаг 1 / хвост шага
            ++j;
        }
        z_function[index] = j;
        left = index;
        right = index + j - 1;
    }
}
```

## 1.3 Полиномиальные хэши

### Опр. 5 (Полиномиальная хэш-функция)

Полиномиальная хэш-функция для строки  $S$  определяется как

$$h(S) = \left( \sum_{i=0}^{|S|-1} S[i] \cdot x^i \right) \bmod p$$

где  $p$  - простое, а  $x \in \mathbb{Z}_p^*$

Заметим, что  $h(Sc)$ , где  $c$  - это символ, который мы добавляем к строке, равен  $\left( \sum_{i=0}^{|S|-1} S[i] \cdot x^i \right) \bmod p + c \cdot x^{|S|} \bmod p = (h(S) + c \cdot x^{|S|}) \bmod p$

### Функция для расчёта полиномиального хэша (C++)

```
long long polynomial_hash(const std::string& text, long long x, long long mod) {
    long long hash = 0;
    long long power = 1; // x^0 = 1 — множитель для самого первого символа
    for (std::size_t index = 0; index < text.size(); ++index) {
        hash = (hash + text[index] * power) % mod;
        power = (power * x) % mod;
    }
    return hash;
}
```

**Утверждение 1** (Хэш подстроки) Пусть  $h[i] = h(S.substr(0, i + 1))$  и  $h[-1] = 0$ , тогда

$$h(S.substr(l, r - l)) = (h[r - 1] - h[l - 1]) \cdot (x^l)^{-1} \bmod p$$

Деление здесь — умножение на обратный к  $x^l$  по модулю  $p$  (он существует, так как  $p$  простое и  $x \in \mathbb{Z}_p^*$ ).

Действительно, для того, чтобы посчитать хэш подстроки  $S[l..r - 1]$ , достаточно из хэша префикса, заканчивающегося в  $r - 1$ , вычесть хэш префикса, заканчивающегося в  $l - 1$ , и всё это поделить на  $x^l$ : каждый следующий  $h[i]$  получается из  $h[i - 1]$  добавлением слагаемого  $S[i] \cdot x^i$  (см. примечание выше), поэтому  $h[r - 1] = h[l - 1] + S[l] \cdot x^l + S[l + 1] \cdot x^{l+1} + \dots + S[r - 1] \cdot x^{r-1}$  — а это как раз хэш подстроки  $S[l..r - 1]$ , домноженный на  $x^l$  (сдвиг степеней из-за того, что подстрока начинается не с позиции 0).

**Алгоритм 4** (Рабина-Карпа) Задача та же, что и в алгоритме КМП

1. Посчитаем хэш паттерна
2. Посчитаем хэши всех префиксов текста
3. Для каждой подстроки текста с длиной паттерна посчитаем её хэш вышеописанным способом и сравним с хэшем паттерна. Если хэши не совпали — подстрока точно не равна паттерну; если совпали — сравниваем посимвольно (хэши могут совпасть и у разных строк — коллизия).

**Теорема 1** (б/д)

Если  $x$  выбирается случайно, то для двух различных строк длины  $|P|$  вероятность коллизии полиномиальной хэш-функции не превосходит  $\frac{|P|-1}{p}$  (разность хэшей — ненулевой многочлен степени не выше  $|P| - 1$ , у него не больше  $|P| - 1$  корней).

**Следствие 1** Выбрав простое  $p > |P|^2$ , получаем, что вероятность коллизии меньше  $\frac{1}{|P|}$ . Проверка каждой коллизии стоит  $O(|P|)$ , а значит, в среднем время работы составит

$$O\left(|P| + |T| \left(1 + |P| \cdot \frac{1}{|P|}\right)\right) = O(|P| + |T|)$$

Примечание автора: многие олпрогеры очень любят хэши за их простоту и быстроту написания и очень любят пытаться записать их везде, где только можно. С первого взгляда это может показаться очень хорошей идеей: действительно, зачем учить множество строковых алгоритмов, если можно выучить одни хэши и с их помощью решать огромное количество задач с практически оптимальной асимптотикой. Однако у такого подхода есть один существенный минус: к любому хэшу можно подобрать контр-тест, на котором возникнет коллизия, и ваше решение перестанет работать. Да, вероятность того, что такое произойдёт очень мала, однако в реальной жизни именно из-за этого программисты предпочитают использовать пусть более сложные, но гораздо более надёжные и стабильные алгоритмы, нежели хэши. Поэтому призываю вас не пытаться всё решить хэшами, а разобраться с остальными алгоритмами и там, где их можно использовать, применять именно их.